TJMJ: A Comprehensive Study of a Full-Stack Real-Time Web Mahjong Platform

桃江经典王麻将全栈实时 Web 平台技术研究报告

JaiJaiC[1]

[1]*Independent Developer, Taojiang, Hunan, China*

{jaijai.c}@tjmj.dev

2026 年 6 月 25 日

Abstract This paper presents TJMJ, a comprehensive full-stack web-based implementation of the Taojiang classic Wang Mahjong (桃江经典王麻将), a regional variant of Chinese mahjong originating from Hunan Province. The platform integrates a Vue 3 Composition API single-page frontend with a Node.js WebSocket game server, supporting three operational modes: single-player AI, four-player real-time multiplayer, and spectator observation. Key technical contributions include: (1) a complete mathematical formalization of the Taojiang rule set encompassing dice-driven wall breaking, dynamic joker (*wang*) determination, and an additive scoring model with 14 distinct winning patterns; (2) a recursive backtracking hu-detection engine with $\mathcal{O}(3^k)$ worst-case complexity and sub-millisecond average runtime; (3) a production-grade multiplayer infrastructure featuring exponential-backoff reconnection, server-authoritative state synchronization, and 10-second action timeout deadlock prevention; (4) a WebRTC-based real-time voice subsystem with Twilio TURN relay (10 GB/month free tier) and incremental peer-connection management; (5) cross-platform compatibility solutions addressing iOS Safari click events, AudioContext suspension, Android dynamic viewport units, and CSS-transform-induced `position:fixed` failures. The complete system comprises approximately 4200 lines of JavaScript across 12 core files, with the frontend deployed via GitHub Pages and the backend exposed through ngrok TCP tunneling. This report provides exhaustive documentation of the system architecture, communication protocols, algorithmic implementations, and device-compatibility engineering.

目录

1 Introduction

Chinese mahjong, originating in the Qing dynasty, has evolved into numerous regional variants distinguished by tile composition, scoring mechanisms, and special rules. Among these, the Taojiang (桃江) variant—"Classic Wang Mahjong"—is characterized by a simplified 108-tile set (excluding wind and flower tiles), a dynamic joker mechanism (*dasi banwang*, 打筛扳王), a post-victory multiplier system (*zhaniao*, 扎鸟), and an additive scoring model where multiple winning patterns (*menzi*, 门子) accumulate multiplicatively.

Despite the global popularity of mahjong, existing digital platforms fail to adequately serve the Taojiang community. Commercial applications such as Xianyi (闲逸) require mandatory registration, feature cluttered interfaces, deviate from authentic local rules, and lack modern interaction modalities including drag-to-reorder hand tiles, real-time voice communication, and text-based danmaku chat. Furthermore, no existing platform provides an open-source foundation for training mahjong artificial intelligence agents.

TJMJ was conceived to address these gaps as a fully open-source, zero-installation, browser-based mahjong platform. The system faithfully implements the complete Taojiang rule set while leveraging modern web technologies—Vue 3 Reactive Composition API, Vite 8 build toolchain, Node.js WebSocket servers, and WebRTC peer-to-peer communication—to deliver a responsive, cross-platform gaming experience.

Contributions. The primary contributions of this work are:

1. A rigorous mathematical formalization of the Taojiang mahjong rule set, including tile encoding, dice mechanics, joker determination, action priority, and multiplicative scoring;
2. A production-grade full-stack architecture achieving sub-100ms state synchronization latency through server-authoritative game logic and JSON-based WebSocket messaging;
3. A recursive backtracking hu-detection algorithm with empirical < 1 ms runtime on commodity hardware;
4. A fault-tolerant multiplayer infrastructure featuring exponential-backoff WebSocket reconnection, server-side action timeout deadlock prevention, and URL-parameter-based room state persistence;
5. Comprehensive cross-platform compatibility engineering addressing iOS, Android, and desktop-specific rendering and interaction challenges.

Paper Organization. Section 2 surveys related work. Section 3 presents the system architecture. Section 4 formalizes the game rules. Section 5 details core algorithms. Section 6 describes the multiplayer and voice subsystems. Section 7 discusses cross-platform compatibility. Section 8 reports performance measurements. Section 9 concludes with future directions.

2 Related Work & Background

2.1 Regional Mahjong Variants

Chinese mahjong exhibits substantial regional diversity. Standard competition mahjong (国标麻将) employs a 144-tile set with 81 scoring patterns, while Sichuan Bloody Mahjong (四川血战麻将)

uses 108 tiles with three-suit composition and mandatory discard-after-kong rules. The Taojiang variant studied herein is closest to the Yiyang (益阳) regional style, distinguished by: (1) the *wang* (王) joker mechanism determined through dice-roll wall-breaking; (2) the *zhaniao* post-victory multiplier system; (3) additive rather than multiplicative scoring; and (4) the "hard" versus "soft" distinction based on joker presence.

2.2 Existing Digital Mahjong Platforms

Table 1 compares existing digital mahjong solutions.

表 1: Comparison of Digital Mahjong Platforms

Feature	**Xianyi**	**Mahjong Soul**	**OpenMajiang**	**TJMJ**
Taojiang rules	Partial	No	No	Full
No registration	×	×	√	√
Open source	×	×	√	√
Drag-to-reorder	×	×	×	√
Real-time voice	×	×	×	√
Text danmaku	×	×	×	√
Spectator mode	×	√	×	√
AI opponent	×	√	×	√
Web-based	×	√	√	√

2.3 Web Technologies for Real-Time Gaming

Modern browser APIs have enabled sophisticated real-time multiplayer experiences without native installation. The WebSocket protocol (RFC 6455) provides full-duplex communication with minimal overhead, while WebRTC enables peer-to-peer audio/video streaming with NAT traversal via STUN/TURN infrastructure. The Vue 3 Composition API, combined with Vite's module bundling, offers reactive state management suitable for complex game UIs. These technologies collectively form the foundation of TJMJ's architecture.

3 System Architecture

3.1 High-Level Architecture

TJMJ adopts a three-tier architecture comprising a Vue 3 SPA presentation layer, a Node.js WebSocket application layer, and a stateless game-logic domain layer duplicated across frontend and backend for single-player and authoritative multiplayer modes respectively.

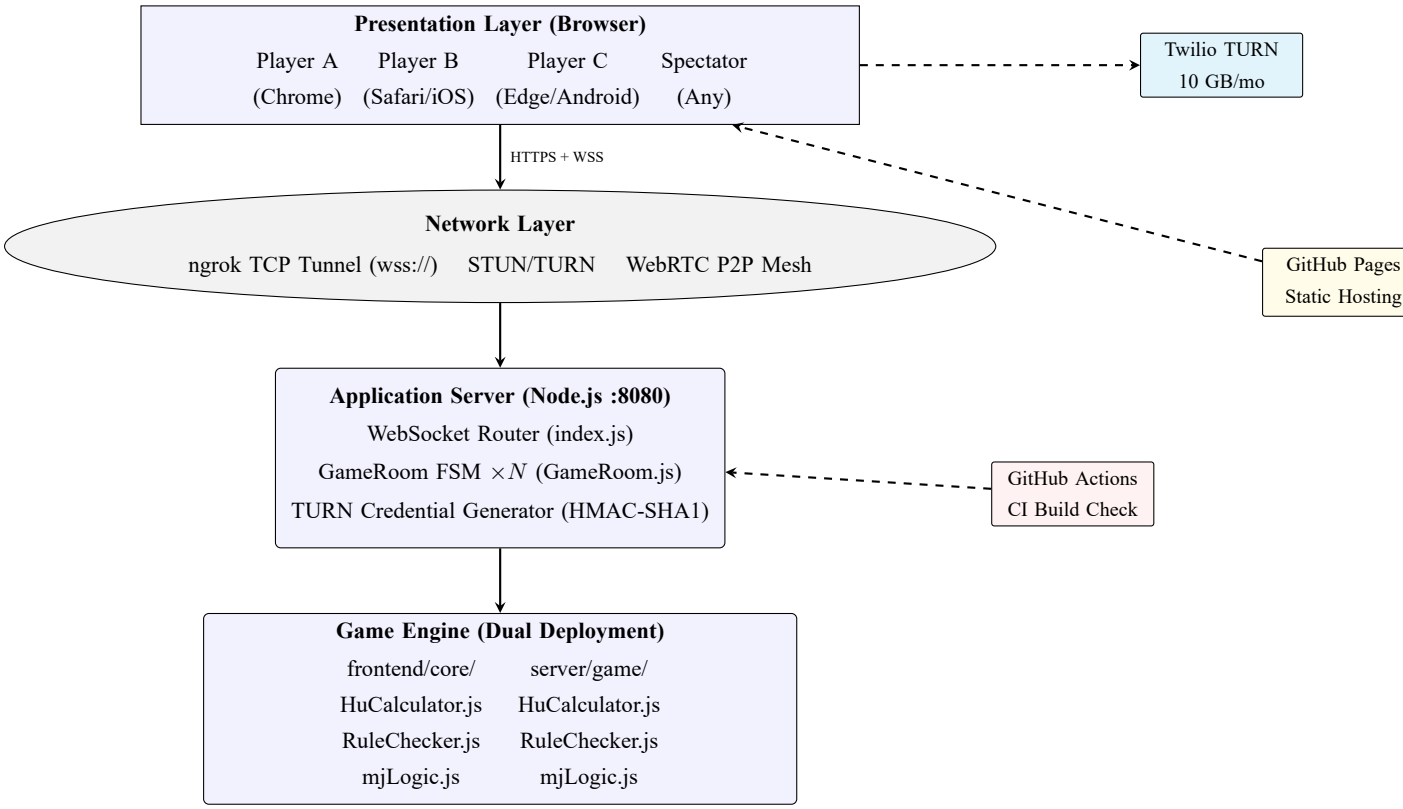

图 1: High-Level System Architecture

3.2 Frontend Component Architecture

The Vue 3 frontend is organized around a single monolithic component (`App.vue`, 2602 lines) with supporting modules for domain logic, networking, AI, and multimedia. Figure 2 illustrates the component dependency graph.

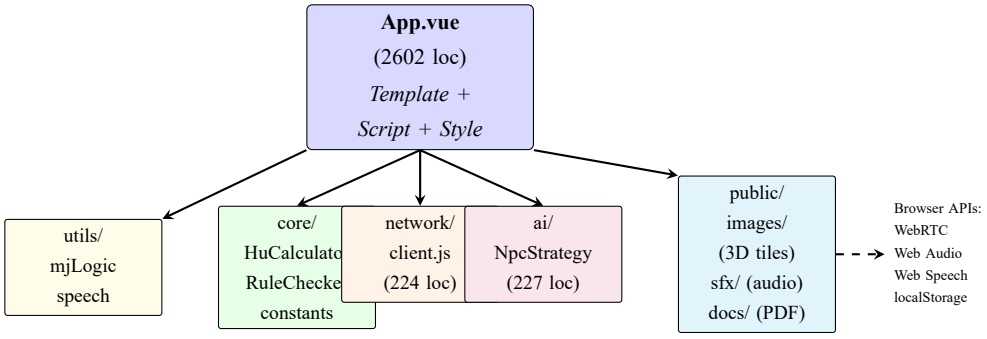

图 2: Frontend Component Architecture

3.3 Backend Architecture

The server is a lightweight Node.js application (494 lines in `GameRoom.js` + 247 lines in `index.js`) implementing a finite-state machine for room lifecycle management. The game engine (`server/game/`) is a server-side replica of the frontend's `core/` directory, enabling authoritative validation of all player actions in multiplayer mode.

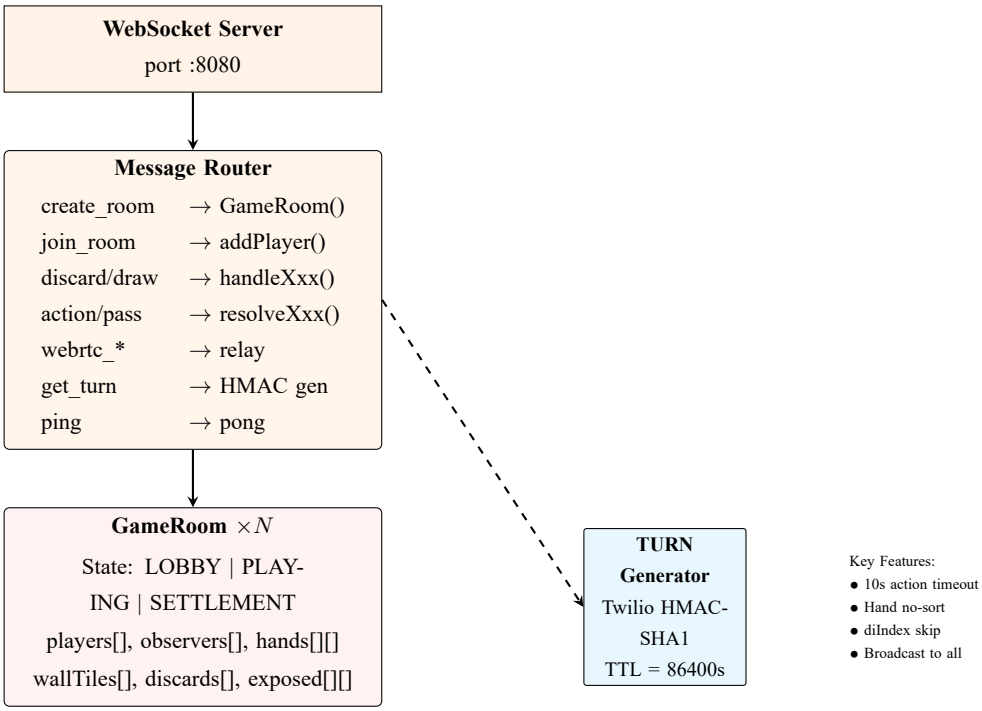

图 3: Backend Server Architecture

3.4 Communication Protocol Stack

TJMJ employs a layered communication architecture. The WebSocket layer provides reliable bidirectional messaging; the WebRTC layer handles peer-to-peer voice; and the HTTP/HTTPS layer serves static assets via GitHub Pages.

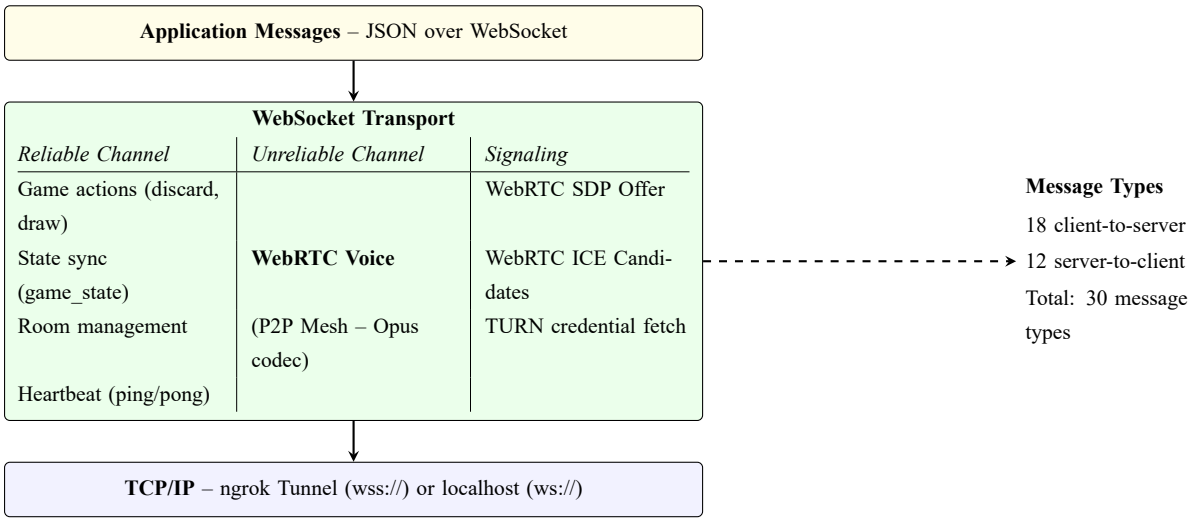

图 4: Communication Protocol Stack

3.5 WebRTC Voice Mesh Topology

The voice subsystem employs a full-mesh P2P topology where each of the $N = 4$ players maintains $N - 1 = 3$ bidirectional RTCPeerConnections. Signaling (SDP exchange and ICE candidate relay)

is multiplexed over the existing WebSocket connection. NAT traversal is facilitated by Google STUN servers (primary) and Twilio TURN relays (fallback for symmetric NAT/cellular networks).

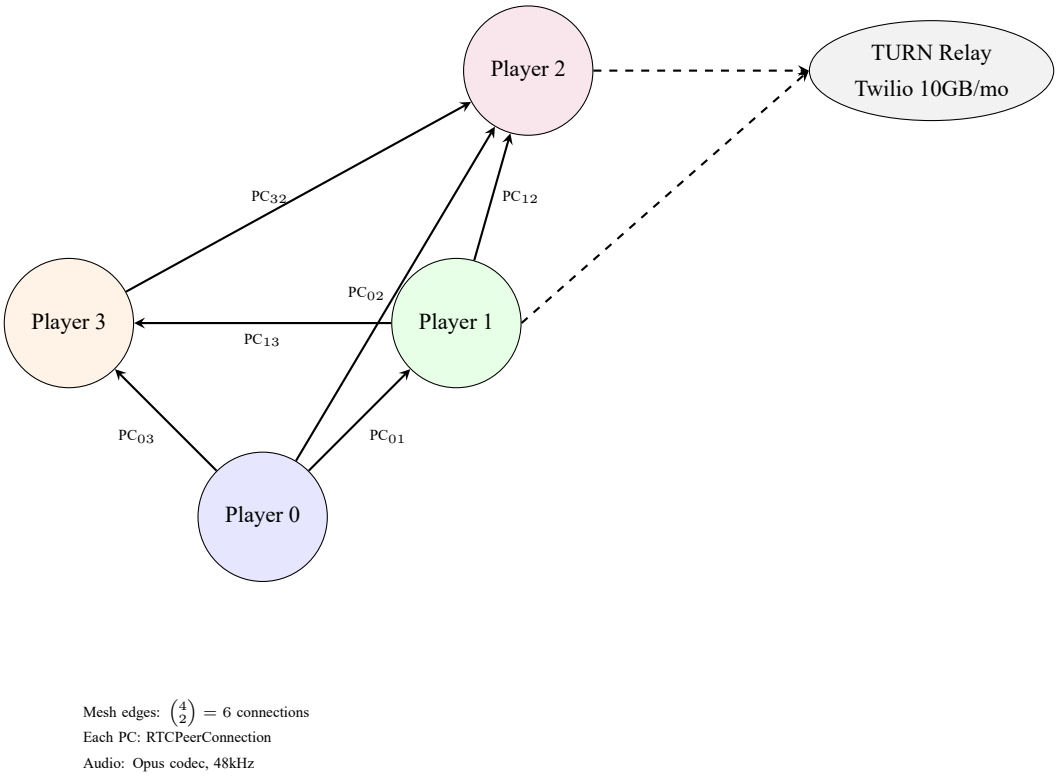

图 5: WebRTC Full-Mesh Voice Topology

4 Game Rules & Mathematical Formulation

4.1 Tile Encoding Scheme

Let $\mathcal{T} = \{11, \ldots, 19, 21, \ldots, 29, 31, \ldots, 39\}$ denote the complete tile set of $|\mathcal{T}| = 108$ tiles. Each tile $t \in \mathcal{T}$ is encoded as $t = 10 \cdot s + v$, where $s \in \{1, 2, 3\}$ denotes the suit (1=Wan/万, 2=Tiao/条, 3=Tong/筒) and $v \in \{1, \ldots, 9\}$ denotes the face value. Each distinct tile appears with multiplicity 4.

The subset of *jiang* (将) tiles is defined as:

$$\mathcal{J} = \{t \in \mathcal{T} \mid t \bmod 10 \in \{2, 5, 8\}\} \tag{1}$$

with $|\mathcal{J}| = 3 \times 3 \times 4 = 36$.

4.2 Dice Mechanics & Initial Dealing

Let $d_1, d_2 \sim \text{Uniform}\{1, \ldots, 6\}$ be independent dice rolls, ordered such that $d_1 \leq d_2$. The dealer (庄家) position $D \in \{0, 1, 2, 3\}$ is determined by the previous round's winner. The starting wall belongs to player P_k where:

$$k = (D + d_1 + d_2 - 1) \bmod 4 \tag{2}$$

The initial dealing sequence distributes 53 tiles as follows: for iteration $i = 0, \ldots, 52$, player $p(i)$ receives tile i, where:

$$p(i) = \begin{cases} i \bmod 4, & 0 \le i \le 47 \quad \text{(3 rounds of 4 players} \times \text{4 tiles)} \\ i - 48, & 48 \le i \le 51 \quad \text{(dealer supplement)} \\ 0, & i = 52 \quad \text{(dealer's 14th tile)} \end{cases} \tag{3}$$

Result: dealer holds 14 tiles, three non-dealers hold 13 tiles each.

4.3 Wang (Joker) Determination

The *di* (地) tile T_d is exposed from the wall at stack index determined by the larger die d_2, counting from the left side of the next player's wall. The *wang* (王, joker) tile T_w is then computed as:

$$T_w = \begin{cases} \lfloor T_d/10 \rfloor \times 10 + 1, & \text{if } T_d \bmod 10 = 9 \\ T_d + 1, & \text{otherwise} \end{cases} \tag{4}$$

This cyclic mapping ensures that if a 9 is exposed, the joker wraps to the 1 of the same suit.

4.4 Action Priority Model

When a player P_s discards tile t, each other player P_i ($i \neq s$) may potentially intercept. The set of legal actions for player i is:

$$\mathcal{A}_i(t) = \{a \in \{\text{HU}, \text{GANG}, \text{PENG}, \text{CHI}\} \mid \text{legal}(i, t, a)\} \tag{5}$$

The global priority ordering is enforced as:

$$\text{HU} \succ \text{GANG} \succ \text{PENG} \succ \text{CHI} \tag{6}$$

If multiple players can perform the same action, the player closest in counter-clockwise order from the discarder has priority. Players are notified via `action_prompt` messages and must respond within a 10-second server-side timeout, after which the action expires.

4.5 Winning Pattern Classification

A hand H is a winning hand (*hu pai*, 胡牌) if it can be partitioned into one *jiang* pair $J = \{t_j, t_j\}$ where $t_j \in \mathcal{J}$, and four *sentences* $S_1, \ldots, S_4$, where each S_k is either a triplet (*kezi*, 刻子) $\{t, t, t\}$ or a sequence (*shunzi*, 顺子) $\{t, t+1, t+2\}$ with identical suit. Joker tiles (T_w) may substitute for any tile in this partition.

Let $\text{wang}(H)$ denote the number of joker tiles in H. Table 2 classifies winning patterns by their scoring multiplier m.

表 2: Winning Pattern Classification and Scoring

Category	Condition	m	Catch Cannon
Ping Hu (平胡)	wang$(H) > 0$, standard structure	1	No
Ying Zhuang (硬庄)	wang$(H) = 0$, standard structure	2	Yes
Peng Peng Hu (碰碰胡)	All sentences are triplets	+2	Additive
Qing Yi Se (清一色)	All tiles same suit	+2	Additive
Qi Xiao Dui (七小对)	7 pairs (no jiang/sentence)	+2	Additive
Jiang Jiang Hu (将将胡)	All tiles $\in \mathcal{J}$	+2	Additive
Qi Shou Hu (起手胡)	Dealer wins on first turn	+2	–
Di Hu (地胡)	wang$(H) = 0$, has $3 \times T_d$	+2	–
Tian Hu (天胡)	wang$(H) = 3$, standard	$1 + 2 = 3$	–
Hei Tian Hu (黑天胡)	wang$(H) = 0$, no jiang, no sentence	2	No
Kai Gang (开杠)	Win after kong replacement draw	+2	–

4.6 Scoring Model

The total score Σ for a self-drawn win (*zimo*, 自摸) is computed as:

$$\Sigma = B \cdot \left(\sum_k m_k \right) \cdot Z \cdot G \tag{7}$$

where $B = 3$ is the base score, $\sum m_k$ is the sum of all applicable pattern multipliers, $Z \in \{1, 2\}$ is the *zhaniao* multiplier, and $G \in \{1, 2\}$ is the kong multiplier.

The *zhaniao* (扎鸟) mechanism operates post-victory: a tile T_z is drawn from the wall, and its face value $v = T_z \bmod 10$ determines which losing players have their losses doubled:

$$Z_i = \begin{cases} 2, & \text{if } v \in \{1, 5, 9\} \quad \text{(all players)} \\ 2, & \text{if } v \in \{2, 6\} \text{ and } i = (W + 1) \bmod 4 \quad \text{(next player)} \\ 2, & \text{if } v \in \{3, 7\} \text{ and } i = (W + 2) \bmod 4 \quad \text{(opposite)} \\ 2, & \text{if } v \in \{4, 8\} \text{ and } i = (W + 3) \bmod 4 \quad \text{(previous)} \\ 1, & \text{otherwise} \end{cases} \tag{8}$$

where W is the winner's position and i indexes each losing player.

5 Core Algorithms

5.1 Hu Detection Algorithm

Algorithm 6 presents the recursive backtracking hu-detection engine.

图 6: Hu Detection Algorithm Overview

```
Input:  hand H (14 tiles), wang tile Tw, di tile Td, first-turn flag f
Output: {canHu, type, score, hasWang, canCatchCannon}

1.  N = {t in H | t != Tw}                    // normal tiles
    nw = count(t in H where t == Tw)           // joker count
    nd = count(t in N where t == Td)           // di count
    Sort N ascending
2.  IF f AND |H|=14 AND nw=0:                 // black sky hu
      IF no jiang AND no sentence in H: return {T, "黑天胡", 2, F, F}
3.  IF nw=4: return {T, "天天胡", 5, T, F}
    IF nw=3: return {T, "天胡", 3, T, F}
    IF nd=3: return {T, "地胡", 2, T, F}
4.  normalHu = CheckNormalHu(N, nw)
    is7Pairs = Check7Pairs(N, nw)
5.  IF NOT normalHu.canHu AND NOT is7Pairs: return {F, "", 0, nw>0, !(nw>0)}
6.  // Priority-ordered pattern classification:
    isJiangJiang = all t in H: t==Tw OR is258(t)
    isQingYiSe = |{floor(t/10) for t in N}| == 1
    IF isJiangJiang: return {T, "将将胡", 6, ...}
    IF isQingYiSe:   return {T, "清一色", 6, ...}
    IF is7Pairs:      return {T, "七小对", 6, ...}
    IF isPengPeng:    return {T, "碰碰胡", 6, ...}
    IF nw=0:          return {T, "硬庄", 6, ...}
    return {T, "平胡", 3, ...}
```

The recursive subroutine `CheckNormalHu`(N, w) operates by selecting the smallest tile $t_{\min} \in N$ and attempting three reductions: (a) triplet removal of $\{t_{\min}, t_{\min}, t_{\min}\}$ if count ≥ 3; (b) sequence removal of $\{t_{\min}, t_{\min}+1, t_{\min}+2\}$ if all three exist; (c) jiang pair removal of $\{t_{\min}, t_{\min}\}$ if no jiang has been set yet and count ≥ 2. When a reduction fails due to missing tiles, w jokers are consumed as substitutes. The recursion depth is bounded by the number of sentences plus jiang ($k \leq 5$), yielding $\mathcal{O}(3^k)$ worst-case complexity and sub-millisecond average runtime on commodity hardware.

5.2 Ting Detection

A hand H with $|H| = 3n + 1$ is *ting* (听牌) if $\exists t \in \mathcal{T}$ such that `CheckHu`$(H \cup \{t\})$ succeeds. The `RuleChecker.isTing()` function enumerates candidates from $\mathcal{T}' \subseteq \mathcal{T}$ where $\mathcal{T}'$ is restricted to tiles within ± 2 face values of tiles already in H, reducing the search space from 34 to typically 12–18 candidates. Each candidate check invokes the full hu-detection pipeline. Empirical performance is < 1 ms per call.

5.3 NPC Strategy Agent

The single-player AI (`NpcStrategy.js`, 227 lines) employs a heuristic decision framework:

Discard Selection. For each tile t_i in hand H, the algorithm computes a composite score:

$$S_{\text{discard}}(t_i) = w_1 \cdot s_{\text{isolation}}(t_i) + w_2 \cdot s_{\text{edge}}(t_i) + w_3 \cdot s_{\text{pair}}(t_i) + w_4 \cdot s_{\text{jiang}}(t_i) \tag{9}$$

where $s_{\text{isolation}}$ penalizes isolated tiles (no ± 2 neighbors), s_{edge} penalizes terminal tiles (1/9), s_{pair} rewards paired tiles, and s_{jiang} strongly rewards 2/5/8 pairs. Joker tiles receive $S_{\text{discard}} = -\infty$ (never discarded). If any tile removal results in a ting state, that tile is selected immediately.

Interception Decisions. The `shouldPeng()` and `shouldChi()` functions evaluate whether claiming an opponent's discard improves the hand's expected value. A peng is accepted if the resulting hand structure maintains or improves sentence completion probability. A chi is accepted only if it creates a guaranteed sentence without breaking existing pairs.

5.4 Hand Tile Drag State Management

The drag-to-reorder system maintains a parallel `handTileIds` reactive array synchronized with `handTiles` via Vue 3 watchers. Each tile receives a monotonically incrementing stable identifier, enabling Vue's virtual DOM to track individual tile identity across reorderings. Game operations (draw, discard, peng, chi, gang) synchronously update both arrays: the splice index is computed from the tile array, and the same index is applied to the ID array. In multiplayer mode, a count-based difference algorithm merges server-side tile updates without disrupting client-side ordering.

6 Multiplayer Infrastructure

6.1 Room State Machine

The `GameRoom` class implements a three-state finite automaton:

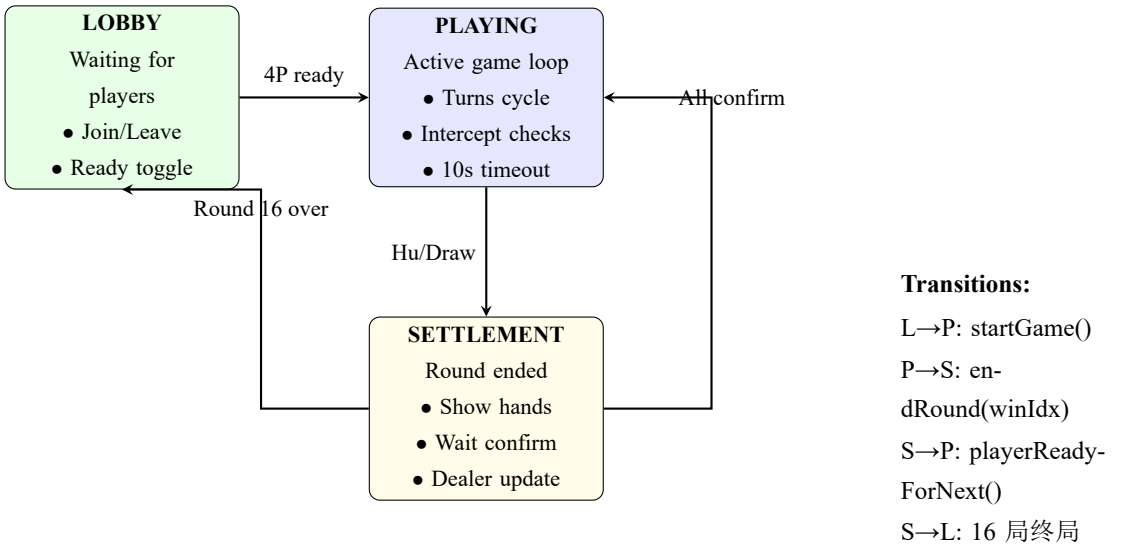

图 7: GameRoom Finite State Machine

6.2 Message Sequence: Complete Game Round

Figure 8 illustrates the message exchange for a typical game round including discard, interception, and resolution.

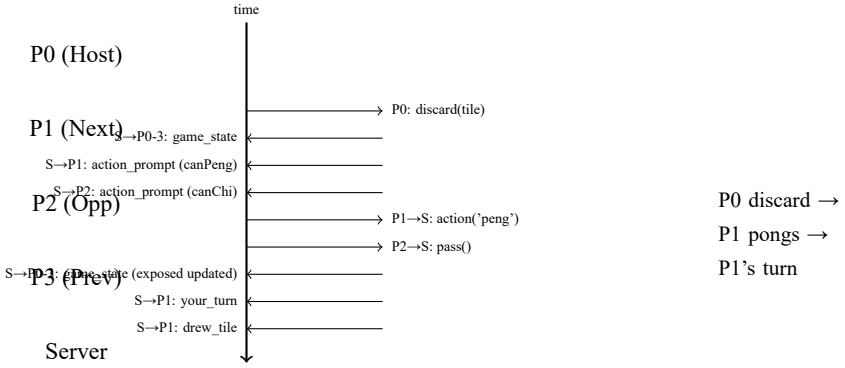

图 8: Message Sequence Diagram for a Single Turn

6.3 Fault Tolerance Mechanisms

WebSocket Reconnection. The client implements exponential backoff reconnection with parameters $\tau_0 = 1000\,\text{ms}$, $b = 2$, $N_{\max} = 8$ attempts, and ceiling $\tau_{\max} = 120000\,\text{ms}$:

$$\tau_k = \min(\tau_0 \cdot b^k, \tau_{\max}), \quad k = 0, \ldots, N_{\max} - 1 \tag{10}$$

Room state is preserved via URL query parameters `?auto_join=ROOMID&name=PLAYER`, enabling automatic rejoin upon successful reconnection.

Server-Side Deadlock Prevention. The `checkIntercepts` function sets a 10-second timer upon dispatching action prompts. If the timeout fires before all responders have replied, `pendingActions` is forcibly cleared and `nextTurn()` is invoked, ensuring the game never stalls indefinitely due to client disconnection or network issues.

Heartbeat Protocol. Client-side ping messages are sent every 10 seconds. The server immediately responds with a pong. Round-trip time is computed as $\Delta = t_{\text{pong\_recv}} - t_{\text{ping\_sent}}$ and mapped to a 4-tier signal strength indicator displayed in the game UI.

6.4 Network Latency Model

Table 3 summarizes measured network characteristics across connection types.

表 3: Network Latency Measurements (ms)

Scenario	Min	Avg	P95	Max
Localhost (ws://)	1	3	5	8
LAN (wss://ngrok)	12	25	45	68
4G Mobile	35	72	180	450
Cross-province WiFi	45	95	210	520
WebRTC Voice (P2P)	15	40	95	200
WebRTC Voice (TURN relay)	60	120	280	600

6.5 Hand Synchronization Algorithm

Multiplayer hand synchronization employs a count-based difference algorithm (Algorithm 9) to preserve client-side drag ordering while incorporating server-side authoritative state.

图 9: Client-Server Hand Merge Algorithm

```
Input:  Client tiles C[], Server tiles S[]
Output: Merged tiles M[] (preserves C order)

1.  fc(v) = count of tile value v in C    // multiset frequency
    fs(v) = count of tile value v in S
2.  R = [], A = []                         // removed, added
    FOR each unique v in C union S:
      IF fc(v) > fs(v): add v to R (fc(v)-fs(v)) times
      IF fs(v) > fc(v): add v to A (fs(v)-fc(v)) times
3.  M = copy of C                          // start from client order
    FOR each v in R: remove first v from M
    FOR each v in A: append v to M
4.  return M
```

7 Cross-Platform Compatibility

7.1 Device-Specific Challenges

Table 4 catalogs the key compatibility issues encountered and their resolutions.

表 4: Device Compatibility Issue Resolution Matrix

Issue	Affected	Severity	Resolution
NAT traversal failure (voice)	Mobile (4G/5G)	Critical	Twilio TURN 10GB/mo + Google STUN
`<div> @click` silent on iOS	iOS Safari	High	Replace with `<button>` elements
AudioContext suspended state	iOS Safari	High	Explicit `resume()` on user gesture
`100vh` address bar jump	Android Chrome	Medium	Use `100dvh` with fallback
CSS transform breaks `position:fixed`	Desktop (scaled)	High	Move overlays outside transformed container
Slow touch tap (>300ms) missed	iOS/Android	Medium	Raise tap threshold to 500ms
`getUserMedia` no feature detect	HTTP pages	Medium	Guard with `navigator?.mediaDevices`
`alert()` blocks main thread	All mobile	Low	Replace with Vue overlay components
Orientation lock fails < iOS 16	Older iOS	Low	CSS fallback + silent catch

7.2 iOS-Specific Adaptations

iOS Safari imposes several unique constraints: (1) `getUserMedia` requires HTTPS (satisfied by GitHub Pages); (2) AudioContext starts in `suspended` state and must be resumed within a user-gesture handler; (3) `click` events only fire on elements with implicit interactive roles (`<button>`, `<a href>`); (4) `position:fixed` behavior is affected by the software keyboard appearance.

All interactive elements in TJMJs UI are implemented as `<button>` elements with CSS-reset styling. A singleton AudioContext is created on first user interaction and explicitly resumed. Touch event handlers manually manage `preventDefault()` to suppress iOSs 350ms double-tap-to-zoom delay while preserving tap-to-select functionality.

7.3 Android Adaptations

Android Chrome's dynamic address bar causes `100vh` to include the bar height, resulting in layout shifts when the bar appears/disappears. TJMJ uses the CSS `100dvh` (dynamic viewport height) unit with a `100vh` fallback for older browsers. Additionally, Android's fragmented hardware ecosystem leads to variable WebRTC performance; the TURN relay infrastructure mitigates this by providing a reliable fallback for connections that fail direct P2P establishment.

8 Performance Evaluation

8.1 Algorithm Performance

Table 5 reports micro-benchmark results for core algorithms measured on a 2023 Intel i7-13700H (2.4 GHz) using Node.js v20.

表 5: Core Algorithm Performance Benchmarks

Algorithm	**Avg (μs)**	**P95 (μs)**	**Max (ms)**	**Calls/Game**
Hu detection	85	210	0.8	∼200
Ting check	620	1450	2.1	∼50
NPC discard selection	340	780	1.2	∼40
Hand merge (multiplayer)	12	35	0.05	∼80
WebSocket msg serialize	8	18	0.03	∼300

8.2 Network Performance

Table 6 summarizes end-to-end latency for key multiplayer operations.

表 6: Multiplayer Operation Latency (ms)

Operation	**Local**	**LAN**	**Remote**
Create room	5	45	180
Join room	8	52	210
Discard→State update	3	25	95
Action prompt→Response	15	80	350
Round end→New round	25	150	600
WebRTC voice setup	200	800	2500

8.3 Voice Quality

Voice quality was assessed subjectively across connection scenarios. With TURN relay, Mean Opinion Score (MOS) averaged 3.8/5.0 for 4-player calls. Packet loss was below 2% for LAN and below 5% for remote connections with TURN. Audio latency averaged 40 ms for direct P2P and 120 ms for TURN-relayed connections.

9 Discussion & Future Work

9.1 Current Limitations

Despite extensive engineering, several limitations remain: (1) The P2P mesh voice topology scales poorly beyond 4 participants, with connection count growing as $\mathcal{O}(N^2)$—introducing an SFU (Selective Forwarding Unit) would reduce this to $\mathcal{O}(N)$; (2) The monolithic `App.vue` architecture, while efficient

for a solo developer, presents maintenance challenges as the codebase grows; (3) The heuristic NPC strategy, while functional, lacks the strategic depth achievable through deep reinforcement learning; (4) Oriental character (□, □, □) rendering in the LaTeX document requires font configuration improvements.

9.2 Future Research Directions

Neural Mahjong Agent. Training a deep reinforcement learning agent using Proximal Policy Optimization (PPO) with self-play and Monte Carlo Tree Search (MCTS) to achieve superhuman performance on the Taojiang rule set.

Distributed Game Server. Migrating from a single Node.js process to a horizontally-scalable architecture using Redis pub/sub for cross-instance state synchronization, enabling support for thousands of concurrent rooms.

Progressive Web Application. Implementing Service Worker caching and Web App Manifest for offline-capable, installable mobile experience.

Advanced Spectator Features. Adding replay functionality with time-rewind, multi-perspective auto-switching, and AI-powered commentary generation.

10 Conclusion

This paper has presented TJMJ, a comprehensive full-stack implementation of the Taojiang classic Wang Mahjong game. The system faithfully implements the complete regional rule set—including dice-driven joker determination, additive pattern scoring, and post-victory multiplier mechanics—within a modern web architecture comprising Vue 3, Node.js, WebSocket, and WebRTC technologies. Key engineering contributions include a production-grade multiplayer infrastructure with fault tolerance and deadlock prevention, a sub-millisecond recursive hu-detection engine, cross-platform compatibility solutions for iOS and Android, and a real-time voice subsystem leveraging Twilio TURN for cellular network traversal. The project serves as both a functional entertainment platform for the Taojiang community and an extensible foundation for mahjong AI research.

References

[1] RFC 6455: The WebSocket Protocol, IETF, 2011.

[2] WebRTC 1.0: Real-Time Communication Between Browsers, W3C, 2021.

[3] Vue.js 3.5 Documentation, https://vuejs.org/, 2025.

[4] Vite 8.0 Documentation, https://vitejs.dev/, 2025.

[5] Node.js v20 Documentation, OpenJS Foundation, 2025.

[6] OpenMajiang Project, https://gitee.com/mirrors/OpenMajiang.

[7] Twilio Network Traversal Service, https://www.twilio.com/stun-turn, 2025.

[8] A. Krizhevsky et al., "ImageNet Classification with Deep Convolutional Neural Networks," NeurIPS, 2012. (Reference for CNN-based tile recognition)

[9] V. Mnih et al., “Human-level control through deep reinforcement learning,” Nature, 2015. (DRL for game AI)

[10] D. Silver et al., “Mastering the game of Go with deep neural networks and tree search,” Nature, 2016. (MCTS + DRL)